

0.49. De plus en plus grand : 1, puis 2

Deux carrés autour du même centre, le second un cran plus grand. Le centre (10, 8) est choisi pour que même le plus grand tienne dans l'écran.

1. Ce qu'il faut comprendre

Deux lignes, une seule différence : la taille. D'abord **1**, puis **2**. Le second carré entoure le premier.

Ce qui est nouveau ici : le placement. Pour des carrés de plus en plus grands, il faut un centre où **ils tiennent tous**. L'écran fait **20 colonnes** (de 0 à 19) et **18 lignes** (de 0 à 17). Son milieu est vers la colonne **10** et la ligne **8**.

Pourquoi (10, 8) est le meilleur endroit : le plus grand carré possible a la taille **8** (17×17). Autour de (10, 8), il va de la colonne $10 - 8 = 2$ à $10 + 8 = 18$, et de la ligne $8 - 8 = 0$ à $8 + 8 = 16$: il tient **entier**. Tous les carrés, de la taille 1 à la taille 8, peuvent donc tourner autour du **même** centre, emboîtés comme des cadres.

Chaque carré revient au centre avant que le suivant commence : le second repart donc bien de (10, 8).

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2113 octets** de cartouche.

1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // LE CENTRE, pour tous les carrés : (10, 8), le milieu de l'écran.
    //
    // l'écran : 20 colonnes (0 à 19), 18 lignes (0 à 17)
    // le plus grand carré (taille 8) autour de (10, 8) :
    //   colonnes  $10 - 8 = 2$  à  $10 + 8 = 18$  → il tient (19 est le bord)
    //   lignes    $8 - 8 = 0$  à  $8 + 8 = 16$  → il tient (17 est le bord)
    // Donc TOUTES les tailles, de 1 à 8, tiennent autour de ce centre.
    //
    // Les deux carrés de ce programme, emboîtés :
    //
    //   colonne :  8  9 10 11 12
    //   ligne 6 :  2  2  2  2  2      1 : le carré de taille 1 (3 × 3)
    //   ligne 7 :  2  1  1  1  2      2 : le carré de taille 2 (5 × 5)
    //   ligne 8 :  2  1 A  1  2
    //   ligne 9 :  2  1  1  1  2
    //   ligne 10 : 2  2  2  2  2

    carre(10, 8, ALPHABET[0], 1, 1, 250, 1); // taille 1 : 3 × 3
    carre(10, 8, ALPHABET[0], 2, 1, 250, 1); // taille 2 : 5 × 5, un cran plus loin

    while (true) {
        image(); // le A est revenu au centre
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est

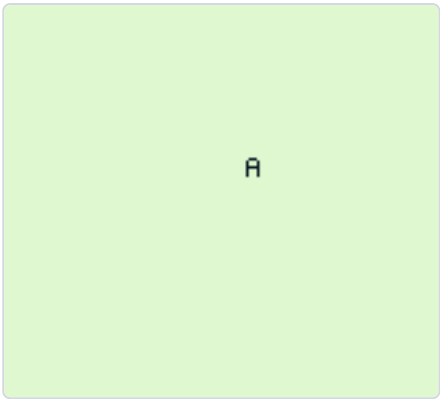
la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un A qui fait un petit tour autour du milieu de l'écran, puis un tour un peu plus grand autour du même point.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le carré de taille 1 : son coin (11, 9)
- ✓ puis celui de taille 2 : son coin (12, 10)
- ✓ le A revient au centre (10, 8) (10,8)

Ensuite : **0.49.1. De plus en plus grand : 1, puis 2 — de base, ailleurs** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++